# claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-aba257a1d8879608a.jsonl`
- SHA-256: `93f65d1f55ed711bd0d0083630aee6d093cdb3265dbc06702f0a76d065f2d474`
- Source modified: `2026-06-22T15:03:44+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`

## Transcript

### 1. user (2026-06-22T15:00:24.576Z)

Review for DEFAULT-OFF / REGRESSION safety. Run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD` and read backend/main.py changes in C:/Users/avidu/Projects/bhi-cloudrun. Verify: (1) when compute_target != cloud_run, get_compute_backend returns None and _maybe_dispatch_remote returns None ? is the in-process segmenter path truly byte-identical (the gate inserts between segment_watermarks and segmenter build ? does it change any state, ordering, or the in-process result)? (2) imports: csv/tempfile added at top (cheap, stdlib) and backend.compute imported LAZILY inside the function (no cloud SDK at module load) ? confirm. (3) does any existing test exercise this region + still pass? (4) is the response dict shape on the cloud success/fail path consistent with the in-process branches the SPA/tests expect? Report concrete findings (file:line). Empty list if none.

### 2. assistant (2026-06-22T15:00:27.282Z)

I'll review the diff in the bhi-cloudrun repo for default-off/regression safety. Let me start by pulling the latest and examining the changes.

### 3. user (2026-06-22T15:00:30.636Z)

```
---BRANCH---
feat/serving-api-cloud-dispatch
---HEAD---
304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF
```

### 4. assistant (2026-06-22T15:00:32.651Z)

Let me look at the diff against origin/master.

### 5. user (2026-06-22T15:00:33.709Z)

```
 backend/main.py                  | 100 ++++++++++++++++++
 tests/test_api_cloud_dispatch.py | 212 ++++++++++++++++++++++++++++++++++++++++
 2 files changed, 312 insertions(+)
```

### 6. user (2026-06-22T15:00:36.316Z)

```
diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
 import os
 import sys
```

```
 import argparse
+import csv
 import json
 import logging
 import shutil
 import socket
+import tempfile
 import threading
 import uuid
 import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
     return FileResponse(path, media_type=media, filename=name)


+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
(cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                    "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                           play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
+    == "cloud_run"``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
+
+    compute = get_compute_backend(global_config)
+    if compute is None:
+        return None  # default-OFF ? run the in-process segmenter below (unchanged)
```

```
+
+    storage_cfg = getattr(global_config, "storage", None)
+
+    def _failed(msg: str) -> Dict[str, Any]:
+        # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+        db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+        return {"video_id": video_id, "status": "failed", "message": msg,
+                "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+    # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+    try:
+        input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+    except ValueError as e:
+        return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+    if input_ref.backend == "local":
+        return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+                       "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+    if not out_root:
+        return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+                       "? that's where the Cloud Run job writes the result.")
+
+    notify("dispatching (cloud_run)")
+    logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+    spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+    try:
+        result = compute.run_infer(spec)  # dispatch the Cloud Run Job + bucket-poll to
completion
+    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+        return _failed(f"cloud dispatch error: {e}")
+    if result.returncode != 0:
+        return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+    notify("ingesting (cloud_run)")
+    storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+    _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+    segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+    _finalize_reingest(video_id, segments, play_wm, cand_wm)
+    return {"video_id": video_id, "status": "success",
+            "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+            "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
         # if the run yields segments; otherwise drop the new partial rows and keep the old.
         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+        # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
```

```
+        # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+        # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+        # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+        # they're dropped on the cloud path in this slice ? a documented follow-up.)
+        remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+        if remote is not None:
+            segmentation_ran = True
+            response = remote
+            return response
         segmenter = segmenter_cls(db_instance, global_config)
         try:
             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
```

**7. assistant (2026-06-22T15:00:38.777Z)**

Now let me read the surrounding context in backend/main.py to verify the gate placement and the in-process path. Let me see the full `_execute_processing` function.

**8. user (2026-06-22T15:00:39.601Z)**

```
525                 "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
526                 "results": [r.model_dump() for r in val_results], "play_segments": segments}
527
528
529     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
530         """The full hash ? validate ? segment ? report pipeline for one video.
531
532         This is the former synchronous /api/process body, unchanged in behavior, now run on
533         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
534         the sync endpoint used to return (UI compatibility); the per-video observability
535         report is still emitted in `finally:` and grafted as response["reports"].
536
537         `progress` is an optional callable(stage: str) used to surface coarse job progress.
538         Raises on unexpected internal errors ? the caller records those as a job failure
539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
545         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
```

```
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
570                     "video_id": video_id,
571                     "status": "failed",
572                     "message": "Video failed validation checks.",
573                     "results": [r.model_dump() for r in val_results],
574                 }
575                 return response
576
577             # 2. Run segmenter & indexer
578             logger.info("[API] Video passed checks. Segmenting and indexing...")
579             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581             segmenter_cls = segmenter_registry.get(seg_name)
582             if not segmenter_cls:
583                 # Submit-time validation already 400s unknown names; this is the defensive
584                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585                 available = list(segmenter_registry.list_available().keys())
586                 response = {
587                     "video_id": video_id,
588                     "status": "failed",
589                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590                     "results": [r.model_dump() for r in val_results],
591                     "play_segments": [],
592                 }
593                 return response
594
595             logger.info(f"[API] Using segmenter: {seg_name}")
596             notify(f"segmenting ({seg_name})")
597             segmenter_actual = seg_name
598             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601             # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602             # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603             # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604             # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605             # they're dropped on the cloud path in this slice ? a documented follow-up.)
606             remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607             if remote is not None:
608                 segmentation_ran = True
609                 response = remote
610                 return response
611             segmenter = segmenter_cls(db_instance, global_config)
```

```
612             try:
613                 result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614             except Exception:
615                 # A raising segmenter must not leave half-written rows: restore the pre-run
616                 # state (same destroy-after-confirm contract as the Failure branch), then
let
617                 # the job worker record the failure.
618                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619                 raise
620             segmentation_ran = True
621
622             if isinstance(result, Failure):
623                 logger.error(f"[API] Segmenter failed: {result.message}")
624                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625                 response = {
626                     "video_id": video_id,
627                     "status": "failed",
628                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
629                     "results": [r.model_dump() for r in val_results],
630                     "play_segments": [],
631                 }
632                 return response
633
634             segments = result.value if hasattr(result, "value") else result
635             notify("finalizing")
636             _finalize_reingest(video_id, segments, play_wm, cand_wm)
637
638             response = {
639                 "video_id": video_id,
640                 "status": "success",
641                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
642                 "results": [r.model_dump() for r in val_results],
643                 "play_segments": segments,
644             }
645             return response
646         finally:
647             # Always emit the per-video observability report (next to the video, or to
648             # report.output_dir). Never raises. Surface the paths in the response.
649             paths = emit_indexer_report(
650                 db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
651                 val_results=val_results, val_context=val_context,
652                 segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
653                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
654             )
```

**9. assistant (2026-06-22T15:00:45.934Z)**

Now let me check the response shape of the success branch. Let me look at how the `finally` block grafts `reports` onto the response, and confirm whether the cloud branch's `return response` still hits the `finally`.

**10. user (2026-06-22T15:00:46.892Z)**

```
654             )
655             if paths and isinstance(response, dict):
656                 response["reports"] = paths
657
658
659     @app.post("/api/process", status_code=202)
660     def process_video(req: ProcessRequest, request: Request):
661         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
662
```

```
663         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
664         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
665         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
666         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
667         """
668         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
669         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
670         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
671         try:
672             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
673         except edge_auth.EdgeAuthError as e:
674             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
675
676         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
677         try:
678             validate_input_path(req.video_path, allowed_root=allowed_root)
679             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
680             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
681             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
682         except ValueError as e:
683             raise HTTPException(status_code=400, detail=str(e)) from e
684         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
685         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
686         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
687         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
688         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
689         # rejects a non-local URI as out-of-root.
690         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
691             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
692         if req.segmenter and not segmenter_registry.get(req.segmenter):
693             available = list(segmenter_registry.list_available().keys())
```

**11. assistant (2026-06-22T15:00:49.312Z)**

Now let me examine the `get_compute_backend` function to verify the default-OFF claim ? that it returns None for every non-cloud_run target.

**12. user (2026-06-22T15:00:50.908Z)**

```
backend/compute/base.py
backend/compute/cloud_run.py
backend/compute/__init__.py
backend/compute/__pycache__/base.cpython-313.pyc
backend/compute/__pycache__/cloud_run.cpython-313.pyc
backend/compute/__pycache__/__init__.cpython-313.pyc
---
__init__.py
__pycache__
base.py
cloud_run.py
```

**13. user (2026-06-22T15:00:53.795Z)**

```
"""Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.

COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
"""

from __future__ import annotations

import os
from typing import Any, Optional

from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
]


def _storage_dict(config: Any) -> dict:
    storage = getattr(config, "storage", None)
    if storage is None:
        return {}
    if hasattr(storage, "model_dump"):
        return storage.model_dump()  # type: ignore[no-any-return]
    return dict(storage) if isinstance(storage, dict) else {}


def get_compute_backend(
    config: Any,
    *,
    run_client: Optional[Any] = None,
    policy: ExecutionPolicy = DEFAULT_POLICY,
    token: Optional[str] = None,
) -> Optional[ComputeBackend]:
    """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
    in-process (the default-OFF path).

    Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
    Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
    / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
    pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
    """
    target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
    if target != "cloud_run":
        return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

    # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
    from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

    job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
    region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
```

```
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

**14. assistant (2026-06-22T15:00:58.504Z)**

The `get_compute_backend` confirms default-OFF: returns `None` unless `compute_target == "cloud_run"`. Now let me verify point (1) ? does the gate insert any state change/ordering issue between `segment_watermarks` and the segmenter build? The gate calls `_maybe_dispatch_remote`, which on the None path returns immediately after importing `backend.compute` and calling `get_compute_backend`. Let me confirm `get_compute_backend` has no side effects on the None path (it doesn't ? just attribute reads).

Let me now check the existing tests that exercise `_execute_processing` to confirm they still pass, and verify the response shape. Let me look at the new test file and find existing process tests.

**15. user (2026-06-22T15:00:59.029Z)**

```
1       """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3       Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4       intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5       GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6       done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7       dispatches; every other target runs the in-process segmenter byte-identically.
8       """
9       import csv
10      import json
11      import os
12
13      import pytest
14
15      pytest.importorskip("httpx")
16      from fastapi.testclient import TestClient
17
18      import backend.compute as compute_pkg
19      import backend.main as m
20      from backend.compute.cloud_run import CloudRunJobClient
21      from backend.config.models import AppConfig
22      from backend.infrastructure.database import Database
23      from backend.infrastructure.execution_policy import ExecutionPolicy
24      from backend.pipeline.validators.base import ValidationResult
25
26      _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
27
28
29      def _pass():
30          return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
31
32
33      class _InlineExecutor:
34          def submit(self, fn, *a, **k):
35              fn(*a, **k)
36
37
38      class _FakeCloudClient(CloudRunJobClient):
```

```
        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""

        def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
            self.calls: list[list[str]] = []
            self.video_id, self.returncode = video_id, returncode
            self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
                                                       (8.74, 11.38, 3, "unforced_error")]

        def run_job(self, job_name, argv, *, region, project=None):
            self.calls.append(list(argv))
            out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
            done_uri = argv[argv.index("--done-uri") + 1]
            if self.returncode == 0:
                outputs = os.path.join(out_uri, "outputs", self.video_id)
                os.makedirs(outputs, exist_ok=True)
                with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
                    json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
                               "status": "success"}, f)
                with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
                    w = csv.writer(f)
                    w.writerow(["start", "end", "shot_count", "ending_reason"])
                    for s, e, sc, er in self.rows:
                        w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
            os.makedirs(os.path.dirname(done_uri), exist_ok=True)
            with open(done_uri, "w", encoding="utf-8") as f:
                json.dump({"video_id": self.video_id, "returncode": self.returncode,
                           "segment_count": len(self.rows), "status": "success"}, f)
            return "exec-1"


    @pytest.fixture
    def cloud_env(tmp_path, monkeypatch):
        """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
bucket read an
        identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate."""
        db = Database(str(tmp_path / "t.db"))
        bucket = tmp_path / "bucket"
        bucket.mkdir()
        cfg = AppConfig.model_validate({
            "deployment": {"compute_target": "cloud_run"},
            "storage": {"backend": "local", "output_root": str(bucket)},
        })
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
        fake_local = tmp_path / "fetched.mp4"
        fake_local.write_bytes(b"FAKEVIDEOBYTES")
        monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg:
str(fake_local))
        monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
        return db, bucket, monkeypatch


    def _inject_fake(monkeypatch, fake):
        """Make backend.compute.get_compute_backend (re-imported inside
_maybe_dispatch_remote) build a
        real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
```

```
token."""
        real = compute_pkg.get_compute_backend
        monkeypatch.setattr(compute_pkg, "get_compute_backend",
                            lambda config: real(config, run_client=fake, policy=_FAST,
token="tok"))


    def _meta(row):
        md = row.get("metadata")
        return json.loads(md) if isinstance(md, str) else (md or {})


    def test_cloud_dispatch_ingests_segments(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))

        assert out["status"] == "success"
        assert len(out["play_segments"]) == 2
        rows = db.query_play_segments(out["video_id"], {})
        assert len(rows) == 2
        metas = [_meta(r) for r in rows]
        assert all(md["source"] == "cloud_run" for md in metas)
        assert {md.get("shot_count") for md in metas} == {5, 3}
        assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}


    def test_cloud_dispatch_builds_correct_spec(cloud_env):
        _db, bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)

        m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))

        assert len(fake.calls) == 1  # exactly one Cloud Run Job execution
        argv = fake.calls[0]
        assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
        assert argv[argv.index("--out-uri") + 1] == str(bucket)
        assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
        joined = " ".join(argv)
        assert "deployment.compute_target=local_gpu" in joined  # container runs INLINE
(never re-dispatch)
        assert "indexing.detector_impl=native" in joined


    def test_cloud_failure_marks_failed_no_rows(cloud_env):
        db, _bucket, monkeypatch = cloud_env
        # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
confirm).
        db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
        fake = _FakeCloudClient(returncode=1)
        _inject_fake(monkeypatch, fake)

        out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
        assert out["status"] == "failed"
        assert "returncode 1" in out["message"]
        assert out["play_segments"] == []
        assert db.query_play_segments(out["video_id"], {}) == []


```

```
    def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
        _db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient()
        _inject_fake(monkeypatch, fake)
        local_path = str(tmp_path / "local_only.mp4")

        out = m._execute_processing(m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid"))
        assert out["status"] == "failed"
        assert "cloud input" in out["message"]
        assert fake.calls == []  # never dispatched a local-only path


    def test_default_off_runs_in_process(tmp_path, monkeypatch):
        """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
runs
        byte-identically (the gate is inert). No Cloud Run dispatch."""
        db = Database(str(tmp_path / "t.db"))
        cfg = AppConfig()  # compute_target defaults to ""
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg)
        monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
{}))
        video = tmp_path / "v.mp4"
        video.write_bytes(b"hashable-bytes")

        class _InProcSeg:
            def __init__(self, db_, cfg_):
                self.db = db_

            def process_video(self, vid, path, *a, **k):
                sid = self.db.add_play_segment(vid, 0.0, 5.0)
                return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]

        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
        out = m._execute_processing(m.ProcessRequest(video_path=str(video),
segmenter="motion"))
        assert out["status"] == "success"
        assert len(out["play_segments"]) == 1
        assert "cloud_run" not in out["message"]


    def test_full_api_process_to_query_via_cloud(cloud_env):
        """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
        to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
        db, _bucket, monkeypatch = cloud_env
        fake = _FakeCloudClient(video_id="vidCloud2")
        _inject_fake(monkeypatch, fake)
        client = TestClient(m.app)

        r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"})
        assert r.status_code == 202
        job_id = r.json()["job_id"]

        job = client.get(f"/api/jobs/{job_id}").json()
        assert job["status"] == "done"
        video_id = job["video_id"]
        assert video_id

        q = client.post("/api/query", json={"video_id": video_id})
        assert q.status_code == 200
        segs = q.json()["play_segments"]
```

```
211         assert len(segs) == 2
212         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
213
```

**16. assistant (2026-06-22T15:01:04.511Z)**

Now let me verify the in-process success branch response shape vs. cloud success branch. The in-process success returns `play_segments: segments` where `segments` comes from `result.value` (the segmenter's return). The cloud branch returns `play_segments: segments` where `segments` comes from `db_instance.query_play_segments(...)`. These could differ in shape. Let me check what `query_play_segments` returns vs what the segmenter returns, and what `_finalize_reingest` expects.

**17. user (2026-06-22T15:01:06.559Z)**

```
backend/main.py:285:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int)
-> None:
backend/main.py:521:    # Only the NEW cloud rows (id > the pre-run watermark);
_finalize_reingest then drops the old ones.
backend/main.py:523:    _finalize_reingest(video_id, segments, play_wm, cand_wm)
backend/main.py:636:        _finalize_reingest(video_id, segments, play_wm, cand_wm)
backend/main.py:795:def query_play_segments(req: QueryRequest):
backend/main.py:1162:            _finalize_reingest(video_id, segments, play_wm, cand_wm)
backend/infrastructure/database.py:424:    def segment_watermarks(self, video_id: str) ->
Tuple[int, int]:
backend/infrastructure/database.py:438:    def prune_segments(self, video_id: str, *, play_upto:
Optional[int] = None,
backend/infrastructure/database.py:459:    def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend/infrastructure/database.py:564:    def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
```

**18. assistant (2026-06-22T15:01:10.682Z)**

I notice a critical detail. The diff at line 618 (in-process raise path) and line 624 (in-process Failure path) call `db_instance.prune_segments(video_id, play_after=..., cand_after=...)`, but the `_failed` helper inside `_maybe_dispatch_remote` (in the diff) calls `db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)`. However, the actual `prune_segments` signature uses keyword `play_upto`/`cand_after`? Let me check the actual signature carefully.

**19. user (2026-06-22T15:01:11.455Z)**

```
424         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
425             """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
426
427             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
428             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
429             """
430             with self._connect() as conn:
431                 cur = conn.cursor()
432                 p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
433                                 (video_id,)).fetchone()[0]
434                 c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
435                                 (video_id,)).fetchone()[0]
436                 return int(p), int(c)
437
438         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439                            play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
```

```
440                             cand_after: Optional[int] = None) -> None:
441             """Delete play/candidate segments by id range (events cascade from
play_segments).
442
443             Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444             successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
445             and keep the OLD ones when the run failed/produced nothing (``*_after``).
446             """
447             with self._connect() as conn:
448                 cur = conn.cursor()
449                 if play_upto is not None:
450                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
451                 if play_after is not None:
452                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
453                 if cand_upto is not None:
454                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
455                 if cand_after is not None:
456                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457                 conn.commit()
458
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
463                     cursor.execute(
464                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474             return self._execute_write(
475                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476                 (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477                 "Failed to add event",
478             )
479
480         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481                                   chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
482                                   stats: Optional[Dict[str, Any]] = None,
483                                   detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
```

**20. assistant (2026-06-22T15:01:15.259Z)**

Good ? `prune_segments(play_after=play_wm, cand_after=cand_wm)` is valid. Now let me examine `_finalize_reingest` to ensure the cloud branch's usage matches.

**21. user (2026-06-22T15:01:16.475Z)**

```
285     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
286         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
287 
288         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
289         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
290         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
291         """
292         if segments:
293             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
294         else:
295             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
296 
297 
298     # --- API Endpoints ---
299     @app.get("/api/config")
300     def get_config():
301         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
302         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
303         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
304         # by tests/test_redact.py + the /api/config redaction test).
305         if global_config is None:
306             return {}
307         return redact_secrets(global_config.model_dump(mode="json"))
308 
309     @app.get("/api/health")
310     def health():
311         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
312         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
313         confirm the engine is reachable and which detector is wired. Never raises."""
314         sport = getattr(global_config, "sport", None)
```

**22. assistant (2026-06-22T15:01:23.519Z)**

Now I understand `_finalize_reingest`. The cloud branch passes `segments` = the new cloud rows (id > play_wm). Since the cloud rows were already inserted via `add_play_segment`, and `_finalize_reingest` with non-empty segments calls `prune_segments(play_upto=play_wm)` ? dropping prior rows. This is consistent.

Now let me verify the response shape concern more carefully. The in-process `play_segments` is what the segmenter returns (e.g. dicts with keys `id`, `start_time`, `end_time`, `duration`, `metadata`). The cloud `play_segments` is `db_instance.query_play_segments(...)`. Let me check the shape of `query_play_segments` output and whether SPA/tests rely on the segmenter's shape specifically.

**23. user (2026-06-22T15:01:24.513Z)**

```
564         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
565             """
566             Dynamically queries play segments based on filters:
567             - duration_min: float
568             - server: string
569             - receiver: string
570             - event_type: string (e.g. smash, foul)
571             """
572             query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
573             params = [video_id]
574 
```

```
575             if filters.get("duration_min"):
576                 query += " AND r.duration >= ?"
577                 params.append(filters["duration_min"])
578 
579             if filters.get("server"):
580                 query += " AND r.server = ?"
581                 params.append(filters["server"])
582 
583             if filters.get("receiver"):
584                 query += " AND r.receiver = ?"
585                 params.append(filters["receiver"])
586 
587             if filters.get("event_type"):
588                 # Check if there is an event of a specific type inside this segment
589                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
590                 params.append(filters["event_type"])
591 
592             segments_list = []
593             with self._connect() as conn:
594                 cursor = conn.cursor()
595                 rows = cursor.execute(query, params).fetchall()
596 
597                 for row in rows:
598                     r_dict = dict(row)
599                     r_id = r_dict["id"]
600                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
601 
602                     # Fetch associated events
603                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
604                     r_dict["events"] = []
605                     for er in event_rows:
606                         e_dict = dict(er)
607                         e_dict["details"] = json.loads(e_dict["details"] or "{}")
608                         r_dict["events"].append(e_dict)
609 
610                     segments_list.append(r_dict)
611 
612             return segments_list
613 
```

**24. assistant (2026-06-22T15:01:28.012Z)**

Now let me check what the in-process segmenter actually returns for `play_segments` to compare the shape. Let me look at how the SPA/compile consumes `play_segments` ? specifically `/api/compile` and the `query` endpoint, and whether they rely on `start_time`/`end_time` keys (DB shape) vs the segmenter's shape.

**25. user (2026-06-22T15:01:29.400Z)**

```
backend/pipeline/segmenters/gemini.py:227:                 if "start_time" in segment:
backend/pipeline/segmenters/gemini.py:229:                         segment["start_time"] =
float(segment["start_time"]) + chunk_start
backend/pipeline/segmenters/gemini.py:380:             start =
_parse_time(segment.get("start_time", 0.0))
backend/pipeline/segmenters/gemini.py:383:                 logger.info(f"Skipping segment
#{idx+1}: start_time ({start:.2f}s) >= end_time ({end:.2f}s)")
backend/pipeline/segmenters/gemini.py:402:                 rejected.append((label, f"start_time
({c['start']:.2f}s) is beyond video duration ({video_duration:.1f}s)"))
backend/pipeline/segmenters/gemini.py:501:                     "start_time": start,
backend/pipeline/segmenters/motion.py:244:                     "start_time": start,
backend/pipeline/segmenters/trajectory_hybrid.py:283:                     start_time=cw["start"],
end_time=cw["end"],
backend/pipeline/segmenters/trajectory_hybrid.py:303:                     "start_time":
```

```
all_candidate_windows[wi]["start"],
backend/pipeline/segmenters/trajectory_hybrid.py:332:                        seg["start_time"] =
float(seg["start_time"]) + w_start
backend/pipeline/segmenters/trajectory_hybrid.py:359:                start =
float(segment["start_time"])
backend/pipeline/segmenters/trajectory_hybrid.py:373:                        "id": segment_id,
"start_time": start, "end_time": end,
backend/pipeline/segmenters/yolo_hybrid.py:253:                    start_time=w["start"],
end_time=w["end"],
backend/pipeline/segmenters/yolo_hybrid.py:292:                    segment["start_time"] =
float(segment["start_time"]) + w_start
backend/pipeline/segmenters/yolo_hybrid.py:327:                start =
float(segment["start_time"])
backend/pipeline/segmenters/yolo_hybrid.py:356:                        "start_time": start,
```

**26. assistant (2026-06-22T15:01:40.128Z)**

The in-process segmenter returns dicts with `start_time`/`end_time`/`id`/`metadata`. The DB `query_play_segments` returns rows with `start_time`/`end_time`/`id`/`duration`/`metadata`/`events` (DB column names). These are compatible ? same key names. The cloud path also adds `events: []` and `server`/`receiver` columns. This is the same shape `/api/query` returns, so the SPA consuming `/api/query` is fine.

Let me verify there's no shape concern with `_finalize_reingest`'s truthiness check on `segments`. In the cloud branch, `segments` is a list, so `if segments:` works. Good.

Now let me check one subtle issue: the cloud branch computes `segments` AFTER calling `_ingest_remote_segments` which inserts rows, but BEFORE `_finalize_reingest` drops the prior rows. The filter is `id > play_wm`. But what if a prior re-ingest left rows with id <= play_wm that are NOT cloud rows? Those are correctly excluded. Good.

Now, a more subtle concern: in the cloud success path, the order is: ingest (insert new rows) ? query new rows ? `_finalize_reingest(segments)` which prunes `play_upto=play_wm` (drops old). That's correct destroy-after-confirm.

Let me now run the relevant tests to confirm they pass, and run the broader process/regression tests.

**27. user (2026-06-22T15:01:44.354Z)**

```
......                                                                   [100%]
6 passed in 2.52s
```

**28. user (2026-06-22T15:01:46.991Z)**

```
test_api_cloud_dispatch.py
test_api_endpoints.py
test_reingest_safety.py
```

**29. user (2026-06-22T15:01:50.422Z)**

```
tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_api_endpoints.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_async_jobs.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_base.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_edge_auth.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_job_worker_loop.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_job_worker_module_seam.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_locale_analytics.cpython-313-pytest-9.0.3.pyc
tests/__pycache__/test_server_entrypoint.cpython-313-pytest-9.0.3.pyc
tests/test_api_cloud_dispatch.py
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_base.py
```

```
tests/test_edge_auth.py
tests/test_job_worker_loop.py
tests/test_job_worker_module_seam.py
tests/test_locale_analytics.py
tests/test_server_entrypoint.py
```

**30. assistant (2026-06-22T15:01:53.389Z)**

These existing tests exercise the changed region (`_execute_processing`, `/api/process`, `_finalize_reingest`). Let me run them all to confirm they still pass ? verifying the in-process path is unchanged.

**31. user (2026-06-22T15:02:22.485Z)**

```
.................................................................... [ 64%]
.....................................                              [100%]
112 passed in 25.12s
```

**32. assistant (2026-06-22T15:02:26.901Z)**

All existing tests in the changed region pass. Now let me verify the import-cost concern (point 2). The diff adds `csv` and `tempfile` at module top (both stdlib, cheap), and imports `backend.compute` lazily inside `_maybe_dispatch_remote`. Let me confirm `backend.compute.__init__` doesn't pull a cloud SDK at module load (it imports `base` and `execution_policy` ? let me confirm `base` is SDK-free, and the cloud SDK is only imported inside `cloud_run` which is itself lazily imported in `get_compute_backend`).

**33. user (2026-06-22T15:02:28.289Z)**

```
"""The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.

COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
(``backend/storage/``): a ``ComputeBackend`` decides **where** the pure DETECT?WINDOW?STITCH
core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
(cloud-serving) ? without the core ever branching on the profile (D1).

The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the worker's
existing in-process path **byte-identical** ? the factory returns ``None`` (= "run inline").
"""

from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Optional


@dataclass(frozen=True)
class ComputeJobSpec:
    """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
    from the SAME input bytes, so the output always lands at ``<out_uri>/outputs/<video_id>/`` ?
    deterministic regardless of where the job ran."""

===CLOUD_RUN_IMPORTS===
18:with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run``
and is
22:from __future__ import annotations
24:import json
25:import logging
26:import os
27:import tempfile
28:import uuid
29:from abc import ABC, abstractmethod
```

```
30:from typing import Any, List, Optional
32:from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33:from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34:from backend.storage import fetch, parse_uri
35:from backend.storage import get_storage
61:    """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI
uses
67:            from google.cloud import run_v2  # type: ignore[attr-defined]
70:                "google-cloud-run is required for CloudRunCompute's real client; install it
on the "
```

**34. assistant (2026-06-22T15:02:36.216Z)**

Confirmed: `google.cloud.run_v2` is imported lazily inside `GoogleCloudRunJobClient` (line 67), not at module load. So no cloud SDK loads at import time. Point (2) verified.

Now let me confirm the lazy import inside the function does not run at module import. The `_maybe_dispatch_remote` function imports `from backend.compute import ...` inside the function body, so it's only triggered at runtime per-call. Good.

Let me verify one concern: the default-OFF path imports `backend.compute` (and thus `backend.compute.base` + `execution_policy`) on EVERY in-process call, even when cloud is off. This is a tiny per-call overhead (already-cached module), not a regression in correctness, but worth noting it executes on the hot path. Let me confirm `backend.compute` is import-cheap (no heavy transitive imports).

**35. user (2026-06-22T15:02:37.732Z)**

```
17:from __future__ import annotations
19:import logging
20:import threading
21:import time as _time
22:from dataclasses import asdict, dataclass
23:from enum import Enum
24:from typing import Callable, Optional, TypeVar
```

**36. assistant (2026-06-22T15:02:48.574Z)**

All stdlib ? `backend.compute` import is cheap. Now let me verify the key default-OFF invariant claim precisely: when `compute_target != cloud_run`, does `_maybe_dispatch_remote` change ANY state before returning None? Looking at the diff:

```python
def _maybe_dispatch_remote(...):
    from backend.compute import ComputeJobSpec, get_compute_backend
    compute = get_compute_backend(global_config)
    if compute is None:
        return None
```

`get_compute_backend(global_config)` on the non-cloud path only reads `config.deployment.compute_target` attribute and returns None ? no DB writes, no notify() calls, no ordering change. The gate sits between `segment_watermarks` (line 600, a read) and the segmenter build (line 611). The watermarks are captured before the gate and used identically by the in-process path afterward. So the in-process path is byte-identical. Point (1) confirmed.

Let me double-check there's no `notify` side effect on the None path ? confirmed, `notify("dispatching...")` only runs after the `compute is None` early-return.

Now let me verify one final correctness concern in the cloud path itself: the `_failed` helper for the local-input rejection runs `prune_segments(play_after=play_wm, cand_after=cand_wm)` even though no rows were written yet on those early-validation failures. This is harmless (deletes id > watermark, of which there are none) but let me confirm there's no scenario where it deletes

legitimate concurrent rows. Since `play_wm` is captured just before, and these failures happen before any insert, pruning `id > play_wm` is a no-op. Safe.

Let me also confirm the `test_default_off_runs_in_process` test actually validates byte-identical behavior ? it does (asserts `cloud_run` not in message, segmenter ran). And let me run a broader smoke to be safe with the full suite collection for the changed module.

**37. user (2026-06-22T15:02:59.646Z)**

```
.................................                                        [100%]
34 passed, 1402 deselected in 8.23s
```

**38. assistant (2026-06-22T15:03:03.775Z)**

All compute/cloud tests pass. Let me do a final check on the `Optional`/`Dict`/`Any` typing imports being available at top of main.py (the new function uses them), and confirm `storage.resolve_input_ref`/`storage.parse_uri`/`storage.fetch` exist as referenced.

**39. user (2026-06-22T15:03:05.878Z)**

```
37:from typing import List, Dict, Any, Optional
61:from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
===STORAGE FNS===
38:def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
59:def acquire_local_input(raw: str, storage_config: Any = None, *,
106:def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
```

**40. assistant (2026-06-22T15:03:09.074Z)**

`typing` imports present, `storage.resolve_input_ref`/`storage.fetch` exist. Let me confirm `storage.parse_uri` is exported (used in `_ingest_remote_segments`).

**41. user (2026-06-22T15:03:10.348Z)**

```
3:URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are imported
20:from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23:    "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
25:    "acquire_local_input", "input_is_local", "resolve_input_ref",
38:def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
48:    return parse_uri(uri)
55:    ``resolve_input_ref``)."""
56:    return resolve_input_ref(raw, storage_config).backend == "local"
70:    ref = resolve_input_ref(raw, sc)
110:    ref = parse_uri(uri)
===config storage model===
311:class StorageConfig(BaseModel):
320:    output_root: str = ""
321:    # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
337:    # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
```

**42. assistant (2026-06-22T15:03:15.692Z)**

All references resolve. `parse_uri` is exported, `output_root` exists on `StorageConfig` (default `""`, correctly guarded by the `_failed` check).

The review is complete. Let me summarize my findings. The change is well-isolated and default-OFF. I found no blockers or regressions. There is one minor observation worth noting: the in-process hot path now imports `backend.compute` and calls `get_compute_backend` on every call (negligible cost, module cached), and a documented functional gap (player_pool/max_frames dropped on the cloud path) that the author already called out in the comment ? not a regression to the default path.

**43. user (2026-06-22T15:03:30.018Z)**

Structured output provided successfully

### 44. assistant (2026-06-22T15:03:44.341Z)

Review complete. Summary of what I verified for DEFAULT-OFF / regression safety in `C:/Users/avidu/Projects/bhi-cloudrun`:

**(1) In-process path is byte-identical when not cloud_run ? CONFIRMED.** The gate at `backend/main.py:606` calls `_maybe_dispatch_remote`, which (`backend/main.py:483`) calls `get_compute_backend(global_config)`. For any target != `cloud_run`, `get_compute_backend` (`backend/compute/__init__.py:47-49`) returns `None` with no side effects ? no DB write, no `notify()`, no ordering change. The gate sits between `segment_watermarks` (`:600`, a pure read) and the segmenter build (`:611`); the watermarks are consumed identically downstream. So the in-process result, state, and ordering are unchanged. `test_default_off_runs_in_process` asserts this.

**(2) Imports ? CONFIRMED.** `csv` + `tempfile` added at module top are stdlib/cheap. `backend.compute` is imported lazily inside `_maybe_dispatch_remote` (`backend/main.py:481`), and it transitively imports only stdlib (`base.py`, `execution_policy.py`). The `google-cloud-run` SDK is lazy-imported inside `GoogleCloudRunJobClient` (`backend/compute/cloud_run.py:67`) and `CloudRunCompute` is lazy-imported only when target==`cloud_run` (`backend/compute/__init__.py:52`). No cloud SDK loads at module import.

**(3) Existing tests in the region still pass ? CONFIRMED.** Ran the tests that exercise `_execute_processing` / `/api/process` / `_finalize_reingest`: `test_api_endpoints`, `test_reingest_safety`, `test_async_jobs`, `test_job_worker_loop`, `test_job_worker_module_seam`, `test_edge_auth`, `test_locale_analytics`, `test_server_entrypoint`, `test_base` ? 112 passed. New `test_api_cloud_dispatch.py` ? 6 passed. All compute/cloud `-k` tests ? 34 passed.

**(4) Response dict shape consistent ? CONFIRMED.** Cloud success (`:521-526`) and fail (`:489-491`) branches carry the same load-bearing keys (`video_id`/`status`/`message`/`results`/`play_segments`) the in-process branches and SPA expect. The cloud `play_segments` come from `query_play_segments` (a superset of the segmenter shape, same key names + `events:[]`), which is exactly what `/api/query` returns ? round-trip proven by `test_full_api_process_to_query_via_cloud`. The `finally` block at `:646-656` still grafts `reports` since the cloud branch `return`s through it.

No blockers or majors. Two `nit`-level observations recorded (both verified safe): the in-process hot path now pays a cached-import + two getattrs per call, and the cloud success branch returns a key-compatible superset shape rather than an identical one. The author already documented the player_pool/max_frames drop on the cloud path as a follow-up (`backend/main.py:604-605`), which does not affect the default path.